Refining Interactions: Enhancing Anisotropy in Graph Neural Networks with Language Semantics (ICME’25)

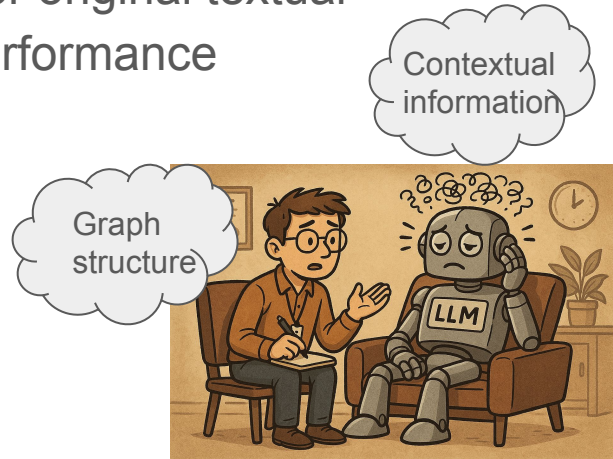
Chloe Kim
Oct 21, 2025

Problem:

- Traditional shallow embedding models capture insufficient context in Text Attribute Graphs (TAGs)
  - Embedding models like Bag of Words, Word2Vec, and TF-IDF have trouble understanding the contextual information of texts effectively, limiting the performance of downstream tasks to some extent.

Note: Recent explorations found feeding graph structure or original textual information of neighbours directly into LLMs -> specific performance improvements.

Leverage LLMs' potential?



Isotropic vs anisotropic GNNS

- Isotropic GNNs = higher efficiency and lower memory overhead
  - E.g., GCN

- Anisotropic GNNS = higher accuracy, but higher overhead
  - E.g., GAT

Inspiration from anisotropic GNNs

- Traditionally = anisotropy manifests in the interactions of embeddings between nodes.
- Embedding level -> natural language level

Solution: LanSAGNN (Language Semantic Anisotropic Graph Neural Network)

- Leverages tailor-made semantic information for node pairs.
- Captures unique interactions within node relationships.
- Dual-layer LLMs finetuning architecture to better align LLMs' outputs without increasing complexity while maintaining strong robustness against interference.

Fig. 1. Traditional GNNs (left) versus Natural Language Level (right) isotropy and anisotropy.

Challenge:

- Ensuring output from LLMs provide PRACTICAL value for graph-related tasks.
- For small-scale, insert multiple trainable GNNs adapter layers between model layers. -> inference speed and structural characteristics of LLMs?? -> approach is not feasible.

Solution for this challenge:

- Dual-layer finetuning architecture tailored to the characteristics of graph data -> improve model performance with a small amount of data.
- Edge filter -> further enhances computational efficiency without sacrificing performance.

Methods

- Focus = node classification problem in TAGs dataset.
- Dataset = G = (V, A, T)
  - V = {v1, v2, …, vn} represents n nodes, each associated with an element ti from set of textual attributes T
  - A = adjacency matrix (graph connections)
- Task = determine class labels of classification task y for all nodes

Methods

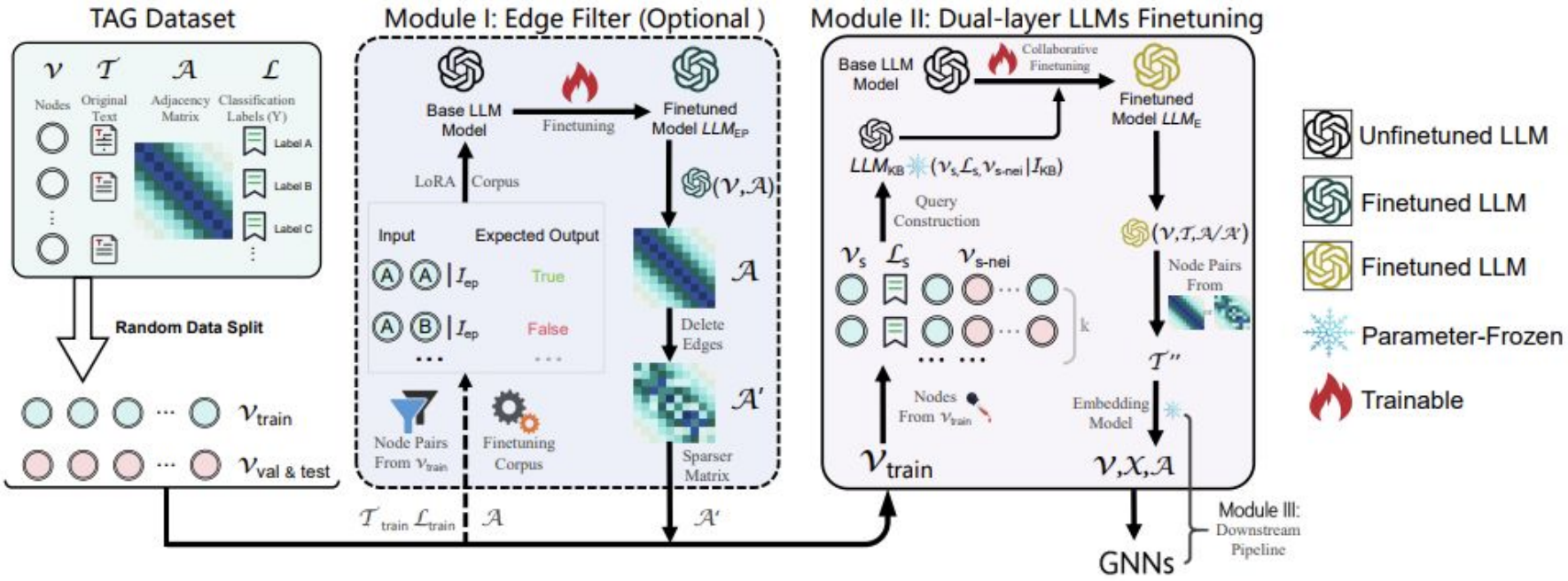


Fig. 2. The overall framework of LanSAGNN.

Methods

$$x_i' = \text{Embedding}(t_i \underset{v_j \in \mathcal{N}(v_i)}{\|} t_{ij}'')$$

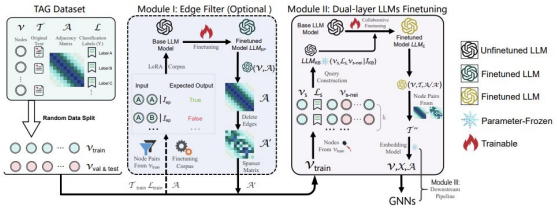


Fig. 2. The overall framework of LanSAGNN.

- Problem: Anisotropic GNNs = accurate but slow (too many edge computations).
  - Fix: Optional Edge Filter removes unimportant edges based on node text similarity.
  - Result: LanSAGNN keeps almost the same accuracy but runs faster and more efficiently — especially on large graphs.
- Dual-layer LLMs
  - Could directly use LLMs to extract info for each node pair to generate customized messages, but doesn't guarantee these messages are beneficial for downstream tasks.
  - New method is more refined and goal-directed
    - Small amount of training data and time are sufficient for LanSAGNN to achieve SOTA performance
    - GPT-3.5-turbo for Knowledge Base LLM, Vicuna1.5-13b as Extraction LLM

Experiment

- Using 4 common TAGs datasets for node classification tasks
  - Cora
  - Pubmed
  - Citeseer
  - Wiki-CS
  - Edges = citation relationships
- Research questions:
  - RQ1. How does LanSAGNN's performance compare to traditional GNNs and methods based on LLMs?
  - RQ2. What are the roles of the various components in LanSAGNN? Does using different LLMs significantly affect the model's performance?
  - RQ3. How do the hyperparameter settings of LanSAGNN affect the model's efficiency? What's the performance of the intervention of optional edge filters?

Experiments cont. (RQ1)

- Evaluate LanSAGNN on 4 TAGs datasets
  - Classical GNN methods
    - GAT
    - GCN
    - GraphSAGE
  - Each dataset's SOTA techniques as baseline comparisons
    - GNN-SOTA (best performing methods on the datasets)
      - ACM-Snowball-3
      - ACM-Snowball-2
      - NHGCN
      - CGT

Experiments cont. (Main Experiment: RQ1)

TABLE I

NODE CLASSIFICATION ACCURACY (%) AND STANDARD DEVIATION UNDER A RANDOM SPLIT OF 60% / 20% / 20% ARE REPORTED, WITH EXPERIMENTAL DATA AVERAGED OVER TEN RUNS.

Category	Method	Cora	Citeseer	Pubmed	Wiki-CS
GNNs	GCN	88.05 ± 0.98	79.26 ± 1.45	88.90 ± 0.32	82.81 ± 1.14
	GraphSAGE	87.87 ± 1.30	79.03 ± 1.03	86.85 ± 0.11	83.76 ± 0.77
	GAT	85.70 ± 0.94	80.30 ± 1.35	83.28 ± 0.12	83.42 ± 0.52
	GNN-SOTA	89.59 ± 1.58	82.07 ± 1.04	91.56 ± 0.50	84.61 ± 0.53
LLM Based	LLM Based Embedding	89.91 ± 0.94	79.94 ± 1.20	92.79 ± 0.31	85.63 ± 0.59
	TAPE	90.22 ± 0.79	81.92 ± 1.32	94.33 ± 0.24	85.83 ± 0.30
	LLM Based A-D & LPA	89.16 ± 0.96	76.84 ± 0.76	94.75 ± 0.36	86.11 ± 0.47
	InstructGLM	90.77 ± 0.52	-	94.62 ± 0.13	-
	GraphEdit	90.90 ± 1.16	81.85 ± 1.42	94.09 ± 0.28	85.85 ± 0.64
Ours	LanSAGNN	**91.24 ± 0.98**	**82.34 ± 1.13**	94.69 ± 0.22	**87.56 ± 0.44**
	LanSAGNN (self loop)	91.02 ± 0.51	82.12 ± 0.95	**95.13 ± 0.25**	87.34 ± 0.42

Experiments cont. (Ablation and Comparative Experiment: RQ2)

- Modest performance increase -> why each module matters

TABLE II
ACCURACY(%) AND STANDARD DEVIATION OF COMPARISON AND ABLATION EXPERIMENTS

Setting	Cora	Citeseer
-w/o Finetune	90.38 ± 0.95	81.24 ± 1.41
-w/o ES	89.83 ± 0.53	80.05 ± 1.00
-w/o OriginText	91.03 ± 1.12	81.51 ± 1.22
LLM_{E}: llama3-8b	91.12 ± 0.85	81.95 ± 0.97
LLM_{E}: vicuna1.5-7b	91.22 ± 0.87	82.01 ± 0.93
LLM: llama3-8b	91.16 ± 0.80	**82.54 ± 1.18**
LanSAGNN	**91.24 ± 0.98**	82.34 ± 1.13

Experiments cont. (Hyperparameter and Optional Edge Filter: RQ3)

k = the number of neighboring nodes considered for each node

TABLE III

ACCURACY(%) AND STANDARD DEVIATION UNDER DIFFERENT k VALUES AND OPTIONAL EDGE FILTER (OEF).

k	Cora	Citeseer	Cora(OEF)	Citeseer(OEF)
1	91.05 ± 1.01	81.84 ± 1.21	-	-
3	91.13 ± 1.03	82.14 ± 1.16	91.09 ± 0.78	82.21 ± 1.34
5	91.31 ± 1.05	82.39 ± 1.07	91.24 ± 0.91	82.51 ± 1.33
10	91.33 ± 0.97	82.67 ± 1.54	91.26 ± 0.76	82.77 ± 0.97
∞	91.37 ± 0.62	82.45 ± 1.27	91.35 ± 1.05	82.84 ± 1.19

Weaknesses

- Modest performance gains (small percentage improvements)
- High complexity from dual LLM fine-tuning
- Expensive to reproduce (requires two RTX A6000 GPUs)
- Unclear runtime trade-offs from dual-LLM process

Conclusion

- **LanSAGNN**: simple, efficient finetuning paradigm
- **Outperforms existing LLM-based methods**
- **Challenges**: interpretability & deployment complexity
- **Future work**: robustness, cost, and case studies